

5. AI for Teaching

Rice Business 2026

Kerry Back
Rice Business

Four Things You Can Now Make

Slides

Decks built from an outline, a paper, or a set of notes — and restyled in one pass

Cases

A fictional company, the numbers, the exhibits, and the teaching note — generated together

RAG chatbots

A tutor grounded in your own course materials, answering with citations

Voiceovers

A narrated video or a self-paced tutorbot made from a deck you already have

Each of these used to be days of work. Each is now a conversation. This session is a tour of how — and every deck in this course was built the way we are about to describe.

Slides

These Slides Are the Example

What they are

- Plain-text **Quarto** files rendered to reveal.js — HTML slides that run in a browser
- One stylesheet controls every deck's look
- Export to PDF or PowerPoint when you need to hand them out

Why text, not PowerPoint

- Claude writes and edits text natively — so it can draft, restyle, and reorganize whole decks
- Change the stylesheet once; all eight decks update
- Version-controlled: every change is tracked and reversible

The slides skill packages this: point Claude at notes, a paper, or an outline and it produces a styled deck with cards, callouts, and section dividers — the components you have been looking at all session.

From Source to Deck



Give it source

An outline, a paper, lecture notes, a transcript



Draft

Claude proposes a slide-by-slide structure



Style

Cards, dividers, diagrams applied from the theme



Export

Present in browser; hand out as PDF or PPTX

You stay in the loop at each step — approving the structure before the polish, and the polish before the export.

Cases

Generating a Case

From concept to case

- Start with the concept a session teaches
- Build a fictional company and a decision that turns on that concept
- Plant one hook for every idea, so a student who understood the material can find them all

What AI drafts

- The company, the numbers, the exhibits, the competing memos
- Realistic artifacts — contracts, data tables, email threads
- The teaching note, cold-call bank, and rubric alongside the case

Your job shifts from writing prose to designing the decision — and checking that every concept has exactly one landmine.

Example: Trinity Freight

Built to test an AI-security session. A freight brokerage's CFO must approve, reject, or redesign an AI quoting agent. Every security concept has one hook in the case.

The decision

Approve a \$5M-opportunity AI agent — or catch why the design is dangerous

The exhibits

Architecture, data inventory, the vendor's security overview, the economics

The hook

One exhibit is an ordinary-looking RFQ email with a prompt injection buried in it

A student who absorbed the session sees the trap in the email. A polished memo that recommends approval and never mentions it has, in effect, approved the breach.

Two Versions, Generated Together

Version A — write the recommendation

- Student plays the CFO and makes the call
- Finds the risks, proposes a redesign, defends the tradeoffs
- Ships with: assignment questions, cold-call bank, grading rubric

Version B — critique the AI's memo

- Student is handed a fluent, confident, subtly wrong AI memo
- Evaluates it rather than writing their own
- Ships with: the AI recommendation, the answer key, the questions

Both versions — the case, the exhibits, the teaching materials, the answer key — came out of the same generation pass. We keep them under version control and edit the parts the model got not-quite-right.

RAG Chatbots

Grounding a Bot in Your Material

One deck: just show it

- A single session's script is small enough to hand the model in full, every time
- No search, no database — the material is simply there
- This is how the tutorbot we will see works

A whole course: too big

- Every deck, reading, and case won't fit in one prompt
- The bot must fetch only the relevant piece for each question
- This is what **RAG** does

Retrieval-Augmented Generation: the model answers from *your* sources instead of its memory — so it stays current with your material and every answer can be checked.

RAG in One Picture

Prepare

Split your documents into chunks, turn each into a numeric “embedding,” and store them in a database

Retrieve

For a question, find the chunks most similar to it and hand them to the model

Answer

The model responds grounded in those chunks — and can cite which document each came from

The Harvard course bots — ChatLTV on 200 documents, an accounting tutor on a whole course — are RAG bots for exactly this reason.

The No-Build Path

Google NotebookLM

- Upload up to 50 sources — PDFs, Google Docs, web links, YouTube
- Answers grounded in your sources, with citations
- Generates study guides and **audio overviews** — a narrated summary
- Free with a Google account

Claude & ChatGPT Projects

- Upload documents to a Project; the assistant searches them across every chat
- RAG without writing any code
- Requires a subscription

For a manageable pile of documents, these are an instant grounded tutor. Build your own only when you need to search thousands of documents, embed it in an app, or control the pipeline.

Voiceovers & Tutorbots

From Deck to Narrated Video

The idea

- Claude drafts a spoken narration script for each slide
- A text-to-speech voice reads it; the audio is synced to the slides
- Out comes an MP4 — a narrated video — plus the transcript

Why bother

- Students who missed class, or want a second pass, get the lecture on demand
- The transcript is searchable and accessible
- You record nothing — and re-generate in minutes when the slide changes

The voiceover skill does exactly this: give it a PDF deck, pick a voice, and it returns the video and transcript.

EchoLeak: Zero-Click Data Theft from Microsoft 365 Copilot

CVE-2025-32711 — June 2025

- 1. Attacker **sends an email** to the victim containing hidden instructions (invisible to the human reader)
- 2. Victim asks Copilot to summarize emails. Copilot reads the hidden instructions.
- 3. Following the injected instructions, **Copilot embeds confidential data into a URL** in its response
- 4. The victim's **browser automatically sends the URL** (as an HTTP request) to the attacker's server
- 5. The attacker **extracts the data from the URL** — zero clicks, zero user awareness

Root cause: Copilot reads untrusted content (emails) **and** has access to private data **and** can render URLs. It combines all three capabilities in one LLM. The defense: separate these roles.

Course tutor

Pause anytime and ask about what you're seeing.

can still see the data; the danger is what happens after it reads. If that same agent also has any outbound channel — it can render a URL, fetch a web page, send an email, or produce a document that later phones home — an injection can instruct it to smuggle the rate table out through that channel, exactly like EchoLeak. Copilot didn't need write access to your mailbox to leak data; it just needed to read private content and have a way to push data out via a URL. So read-only access closes the "destructive action" leg but does nothing about the "outbound channel" leg of the lethal trifecta. To actually stop exfiltration you need to remove or restrict the outbound channel itself — no web rendering, no email sending, whitelisted recipients, sanitized output — not just make the data access read-only.



16 / 40

Play

Ask a question

Resume

☒ Autoplay

slide 16 / 40

Ask about this slide...

Send

☐ Read answers aloud

The slides play with AI voiceover and advance on their own. The student can pause at any moment and ask a

How the Tutorbot Is Built

Narrated deck

Each slide carries a spoken script, pre-generated to one audio clip per slide

Player

Slides plus synced audio; pause, ask, resume; auto-advances when a clip ends

Tutor

Claude answers with the full narration as context, and knows which slide you are on

Question log

Every question saved with its slide number — next class's cold-call material

Anchored, not caged: the tutor is grounded in the deck's script but free to draw on general knowledge — and it signposts when it goes beyond the session.

Made Repeatable: a Skill

The whole process is packaged as a Claude skill, `tutorbot-builder`. Point it at a deck and it produces the narrated tutorbot — deck, narration, audio, player, and tutor backend — no code required.

Any deck in

PowerPoint, PDF, or
reveal.js / Quarto

Convert if needed

PowerPoint and PDF
are turned into slides
first; Claude reads each
to draft narration

Tutorbot out

Narrated, interruptible,
self-paced — ready to
assign before class

A skill is codified expertise you write once and reuse — the subject of our next-to-last session. It is shareable: any instructor can install it and convert their own decks.

The Payoff

Slides, cases, a grounded tutor, a narrated lecture — each was once days of work and is now a conversation. The scarce thing is no longer *making* the content. It is **deciding what to make and judging whether it is good.**

Breakout

Questions to Discuss

- Which of your existing decks would you turn into a narrated tutorbot first?
- What concept in your course would make the best case — and what artifact would carry its central hook?
- Do you have a document pile (readings, notes, past exams) that a grounded chatbot could answer from?
- Where would AI-drafted content still need your hand before it reached students?